

110550022 LAB4

Introduction

This report explains the code and shows the experiment results of lab 4.

Implementation

Training

```
def forward(self, x, x_next, p_next):
    x_transform = self.frame_transformation(x)
    x_next_transform = self.frame_transformation(x_next)
    p_next_transform = self.label_transformation(p_next)

    z, mu, logvar = self.Gaussian_Predictor(x_next_transform, p_next_transform)

    decoder_out = self.Decoder_Fusion(x_transform, p_next_transform, z)
    x_next_pred = self.Generator(decoder_out)

    return mu, logvar, x_next_pred
```

The forward function is implemented as shown in the graphs of the lab specs.

```
def training_one_step(self, img, label, adapt_TeacherForcing, beta):
    self.optim.zero_grad()

    mse_loss = 0.0
    kl_loss = 0.0

    x = img[:, 0]
    len_to_predict = img.shape[1] - 1

    for i in range(len_to_predict):
        if adapt_TeacherForcing:
            x = img[:, i]
            x_next = img[:, i + 1]
            p_next = label[:, i + 1]
            mu, logvar, x_next_pred = self.forward(x, x_next, p_next)
            kl_loss += kl_criterion(mu, logvar, self.batch_size)
            mse_loss += self.mse_criterion(x_next_pred, x_next)

        x = x_next_pred

    loss = mse_loss + beta * kl_loss
    loss.backward()
    self.optimizer_step()

    return loss
```

Given a frame sequence of length n and the first frame as the initial condition, at each training step the model needs to predict $n - 1$ subsequent frames. X_n , X_{n+1} and P_{n+1} are passed into the forward function to obtain μ , log variance and the model's prediction of X_{n+1} . The μ and log variance values are then used to compute the KL divergence loss and we also calculate the MSE loss between the prediction and ground truth of X_{n+1} . At the end of each iteration, the prediction is set as the X_n for the next iteration, and if teacher forcing is set to true, it is overwritten with the ground truth frame.

After predicting all frames, kl annealing is applied by assigned the weight beta to the kl loss term. Originally, I chose to normalize the final loss by the length of the sequence, but since all training sequences have a common length of 16, I dropped the normalization step to avoid losing precision.

Evaluation (Testing)

The inference and evaluation codes are nearly identical, so I will only go over evaluation here.

```
def forward_eval(self, x, p_next):
    x_transform = self.frame_transformation(x)
    p_next_transform = self.label_transformation(p_next)

    z = torch.randn((1, self.args.N_dim, self.args.frame_H, self.args.frame_W), device=self.args.device)

    decoder_out = self.Decoder_Fusion(x_transform, p_next_transform, z)
    x_next_pred = self.Generator(decoder_out)

    return x_next_pred
```

Another forward function was implemented because during inference, instead of the posterior distribution predicted by the Gaussian predictor, z is randomly sampled from a normal distribution.

```
def val_one_step(self, img, label):
    mse_loss = 0.0

    x = img[:, 0]
    len_to_predict = img.shape[1] - 1

    for i in range(len_to_predict):
        x_next = img[:, i + 1]
        p_next = label[:, i + 1]

        x_next_pred = self.forward_eval(x, p_next)
        mse_loss += self.mse_criterion(x_next_pred, x_next)

        x = x_next_pred

    return mse_loss
```

The `val_one_step` is simpler than its training counterpart since we no longer need to keep track of kl loss or apply teacher forcing. MSE loss is chosen as the evaluation criterion since PSNR is basically MSE with extra steps.

Reparameterization

During training, the model maps the input x to a distribution over latent variables z and samples $z \sim q(z|x)$ to produce the output of the Gaussian predictor.

However, the sampling operation is non-differentiable, so we will have to reparameterize $N(\mu, \sigma^2)$.

To produce a distribution that matches $z \sim N(\mu, \sigma^2)$, we can multiply $\epsilon \sim N(0, 1)$ by σ and shift it by μ , i.e.

$$Z = \mu + \sigma * \epsilon$$

```
def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    epsilon = torch.randn_like(std)
    return mu + std * epsilon
```

$$\text{Log}(\text{var}) = \log(\text{std}^2) = 2\log(\text{std})$$

$$\text{Log}(\text{std}) = 0.5 * \log(\text{var}), \text{std} = \exp(0.5 * \log(\text{var}))$$

Teacher Forcing

If the `adapt_TeacherForcing` parameter of `training_one_step` is set to `true`, the `x` at the start of the iteration will use the ground truth frame instead of the prediction from the previous iteration.

```
if adapt_TeacherForcing:  
    x = img[:, i]
```

```
def teacher_forcing_ratio_update(self):  
    if self.current_epoch > self.tfr_sde:  
        self.tfr = max(0, self.tfr - self.tfr_d_step)
```

If the current training epoch is after `tfr_sde`, every epoch afterwards will deduct `tfr_d_step` from the teacher forcing ratio, down to a minimum of 0.

KL annealing ratio

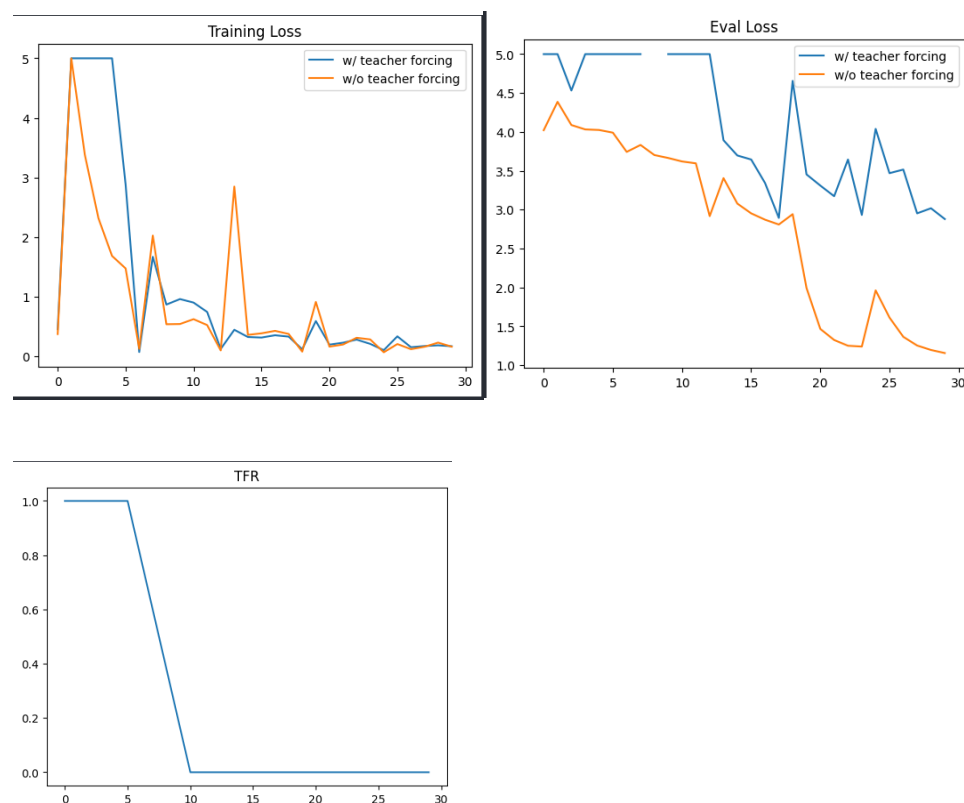
```
class kl_annealing():  
    def __init__(self, args, current_epoch=0):  
        # TODO  
        self.current_epoch = current_epoch  
        self.use_schedule = True  
  
        if args.kl_anneal_type == "Cyclical":  
            self.schedule = self.frange_cycle_linear(args.num_epoch,  
                                                    n_cycle=args.kl_anneal_cycle,  
                                                    ratio=args.kl_anneal_ratio)  
        elif args.kl_anneal_type == "Monotonic":  
            self.schedule = self.frange_cycle_linear(args.num_epoch, ratio=args.kl_anneal_ratio)  
        elif args.kl_anneal_type == "Without":  
            self.use_schedule = False  
        else:  
            raise Exception("Unsupported annealing strategy")  
  
    def update(self):  
        self.current_epoch += 1  
  
    def get_beta(self):  
        return self.schedule[self.current_epoch] if self.use_schedule else 1  
  
    def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0, n_cycle=1, ratio=1):  
        T = n_iter / n_cycle  
        inc_T = T * ratio  
        m = (stop - start) / inc_T  
        betas = [stop] * n_iter  
  
        T = int(T)  
        inc_T = int(inc_T)  
        for cycle in range(n_cycle):  
            accum = start  
            for i in range(inc_T):  
                betas[cycle * T + i] = accum  
                accum += m  
  
        return betas
```

To work out the linear annealing schedule, we can first calculate the slope of the annealing section. Next, we initialize an array with max beta values so once the

annealing section is over, subsequent checks can be skipped until the next cycle. At each step the beta is increased by m for $T * \text{ratio}$ steps, and this is repeated for n_{cycle} cycles. The beta schedule is stored in an array on initialization and accessed using `self.current_epoch`.

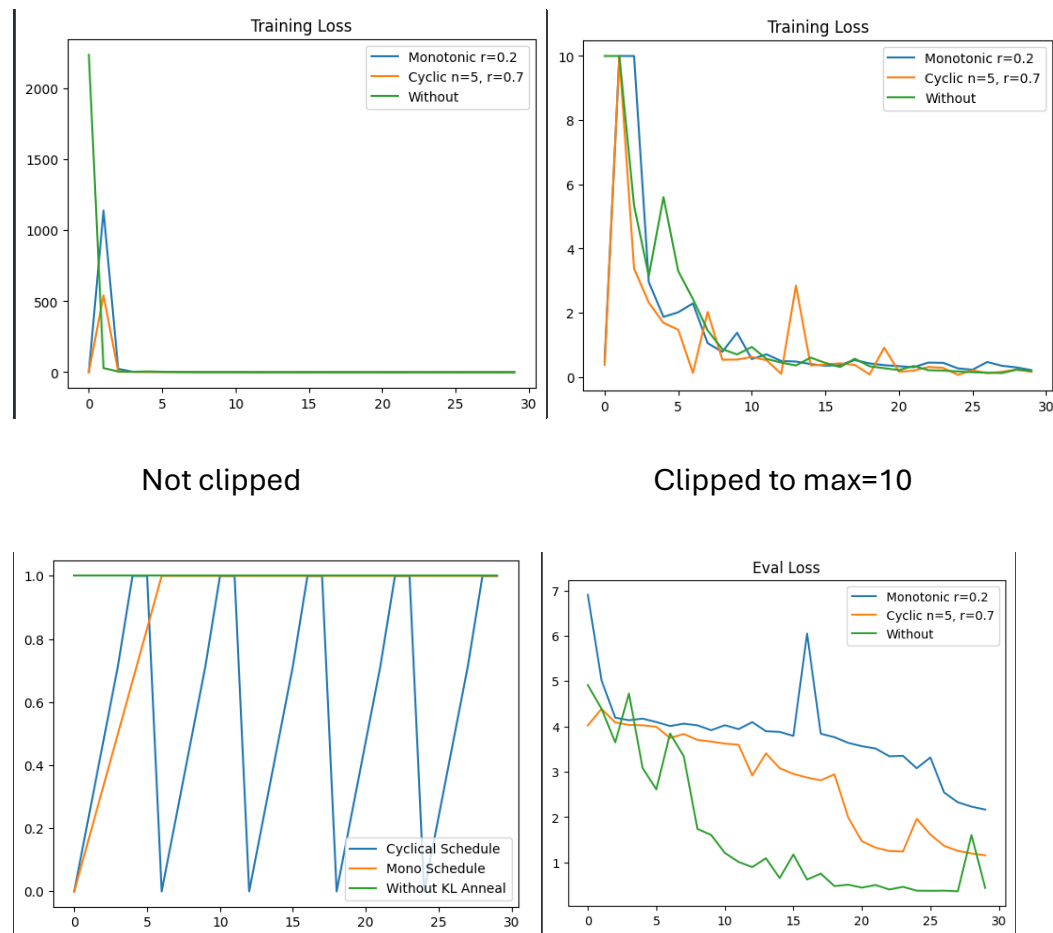
Analysis & Discussion

Teacher Forcing



Tests show that teacher forcing negatively impacts model accuracy for this specific task. Other sources such as [deeplearning.org book, 10, p.378](#) also suggests that “The disadvantage of strict teacher forcing arises if the network is going to be later used in a closed-loop mode, with the network outputs (or samples from the output distribution) fed back as input. In this case, the fed-back inputs that the network sees during training could be quite different from the kind of inputs that it will see at test time.”, which matches with our use case.

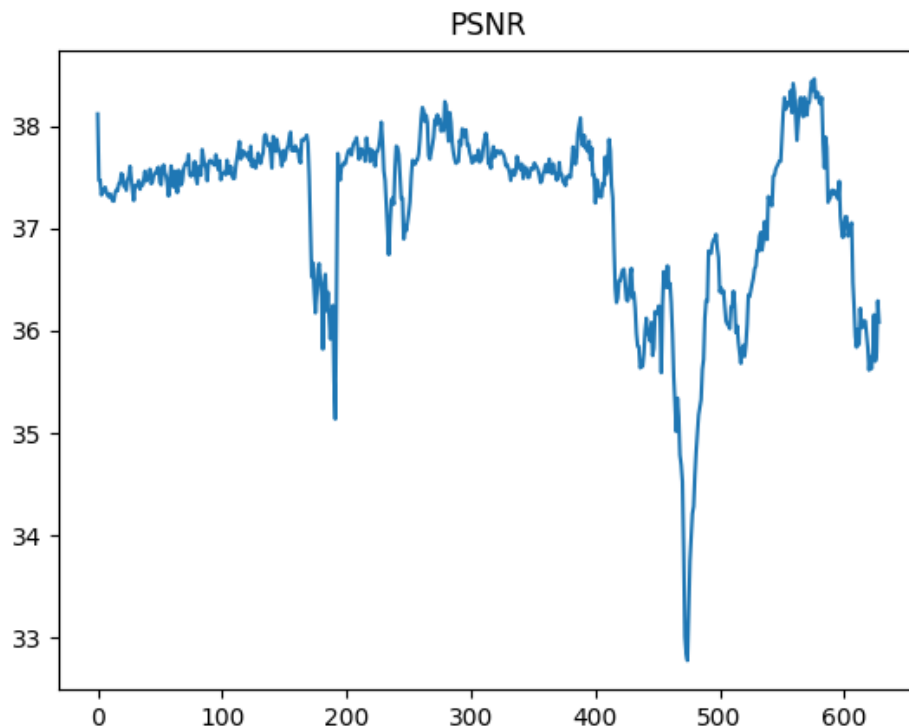
Different KL annealing strategies while training



At the start of the training process, the model that uses no kl annealing has the highest loss since the initial distribution is very far from the standard normal prior, leading to an extremely large kld loss. Monotonic and cyclic scheduling both use low starting betas which mitigates the dominance of the kld loss term. And after zooming in, we can also see spikes of loss values for the cyclic schedule.

However, looking at the evaluation loss trend, the model that was trained using no kl annealing vastly outperforms both methods. This might be due to the simplicity of the dataset.

PSNR on the validation set



PSNR graph of the validation sequence using the best model

Other training strategy analysis

Larger feature dimensions

Since the testing / evaluation set and the training set are actually pretty similar and the dimensions are small, I found that using larger feature dimensions to “memorize” the dataset help with model accuracy.

More fast train epochs

During training, I used only 40% (fast_training) of the dataset for the first 100 epochs out of 150~160 and found that it helped the model converge a lot better. The final 50 epochs on the full dataset should help the model generalize a bit better.

Retraining on nan outputs

Sometimes the decoder outputs nan values, probably due to poor initialization, and what I did was to repeat the first couple of epochs until it stops producing nan values and afterwards the training seem to stabilize.